

Audio Processing in NLP

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026

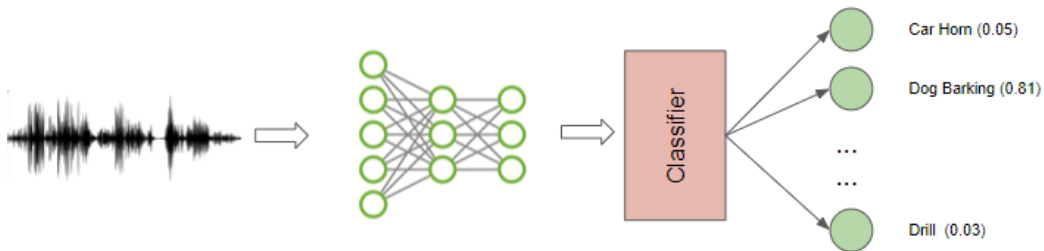


Image source: https://miro.medium.com/v2/resize:fit:1400/1*3MadXYXhtHr-XZNP5-1GXg.png

1. Introduction & Motivation
2. Learning Outcomes
3. Fundamentals of Sound
4. Pre-processing Pipeline
5. STFT & Spectrograms
 1. Fourier Transforms
 2. Short-Time Fourier Transform (STFT)
 3. Spectrograms
6. Mel-Scale & Filterbanks
 1. Perceptual Frequency Scales
 2. Mel Scale
 3. Mel Filterbanks

- 4. Choosing Features
- 5. Mel-Spectrograms
- 6. Log-Mel Features
- 7. MFCC Computation
- 7. Alternative Features
- 8. Automatic Speech Recognition (ASR)
- 9. Connectionist Temporal Classification (CTC)
- 10. Seq2Seq ASR with Attention
- 11. RNN-Transducer (RNN-T)
- 12. Transformer & Conformer in ASR
- 13. Self-Supervised Learning: Wav2Vec 2.0
- 14. Fine-tuning for ASR with Wav2Vec 2.0

- 15. Conformer-CTC: Non-autoregressive ASR
- 16. Spoken Language Understanding (SLU)
- 17. Advanced Tasks: Diarization & Emotion Recognition
- 18. Text-to-Speech (TTS) & Vocoder
- 19. Evaluation
- 20. Challenges & Current Research
- 21. Summary
- 22. References

Speech is the **most natural form of communication**.

What: Where speech interfaces are essential.

- ▶ Voice assistants (e.g., Amazon Echo, Siri, Google Assistant)
- ▶ Accessibility: captions, screen readers
- ▶ Hands-free operation (cars, IoT)
- ▶ Multilingual interaction & translation

Why: Speech enables more intuitive and inclusive interaction.

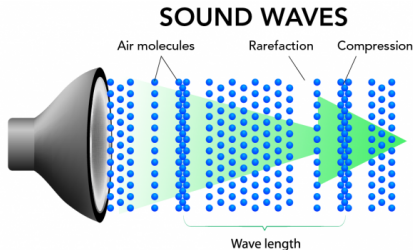
Why audio needs its own toolkit.

- ▶ Audio data is abundant and diverse, encompassing speech, music, and environmental sounds.
- ▶ Text-oriented methods do not apply directly: audio arrives as a continuous waveform with no tokens, no word boundaries, and thousands of samples per second.
- ▶ Many NLP systems now start from speech, not just text (e.g., live captions, real-time translation).
- ▶ The signal carries information text discards: speaker identity, prosody, emotion, and overlapping speakers.

By the end of this session, you will be able to:

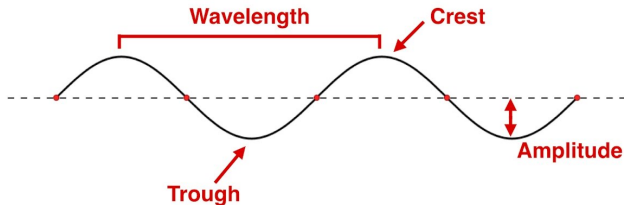
- ▶ Explain how a sound wave becomes a digital signal (sampling, quantization, bitrate).
- ▶ Explain how raw audio becomes features (MFCC, Mel/Log-Mel).
- ▶ Read & reason about spectrograms (STFT, mel scale).
- ▶ Compare the main ASR architectures: CTC, attention-based encoder-decoder, transducers.
- ▶ Describe the Text-to-Speech (TTS) pipeline and where its difficulty lies.
- ▶ Connect features $\rightarrow$ ASR/NLU/TTS.
- ▶ Evaluate a system with the right metric (WER, CER, DER, MOS).
- ▶ Recognize the challenges and future directions in audio processing for NLP.

Fundamentals of Sound



Source: Let's Talk Science, "What is Sound and How do we Hear it?"

- ▶ Sound is a type of energy produced by vibrating objects.
- ▶ It travels through air (or other media) as waves.
- ▶ Sound waves are longitudinal waves, meaning the particle displacement is parallel to the direction of wave propagation.



Source: "Sound and Audio," COMD1112, OpenLab at City Tech, CUNY

- ▶ **Waveform** $x(t)$: The shape of the sound wave, representing how air pressure changes over time.
- ▶ **Frequency**: Number of cycles (oscillations) per second, measured in Hertz (Hz). Determines the pitch of the sound.
- ▶ **Amplitude**: The height of the wave, representing the loudness or intensity of the sound.

- ▶ **Pitch/Formants:** The perceived frequency of a sound and its resonant frequencies, respectively.

To convert an analog signal into a digital one, two main steps are used:

- ▶ **Sampling:** The process of taking measurements of the continuous signal at regular time intervals.
- ▶ **Quantization:** The process of rounding off the amplitude of each sample to the nearest value on a finite scale.

The result is a **discrete signal**, denoted as $x[n]$, which is a sequence of numbers representing the original analog signal.

Why do we digitize signals?

- ▶ **Computer Processing:** Computers can only understand and manipulate discrete numerical values.
- ▶ **Reproducibility:** Digital signals can be copied and transmitted without any loss of quality.
- ▶ **Processability:** Digital signals are easier to process and manipulate using various algorithms for tasks such as noise reduction, filtering, and compression.

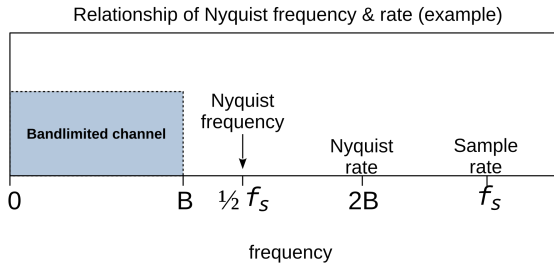
- ▶ **Sampling Rate** (f_s): The number of samples taken per second, measured in Hertz (Hz) or kilohertz (kHz). Common values include 16 kHz for speech and 44.1 kHz for music.
- ▶ **Nyquist–Shannon Sampling Theorem**: States that to perfectly reconstruct a continuous signal, the sampling rate (f_s) must be at least twice the maximum frequency ($f_{\max}$) of the signal.

$$f_s \geq 2f_{\max}$$

Failing to meet this condition can cause **aliasing**, where high frequencies are incorrectly represented as lower frequencies. The Nyquist limit, also known as the Nyquist frequency, is defined as half of the sampling rate:

$$f_{\text{Nyq}} = \frac{f_s}{2}$$

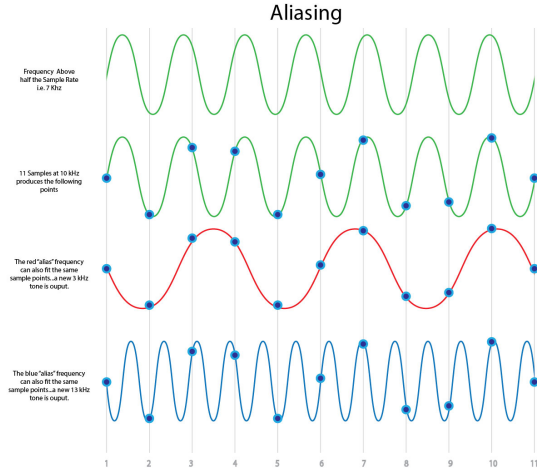
- ▶ **Quantization**: The process of mapping continuous amplitude values to a finite set of discrete levels. This process introduces **quantization noise**, which is a form of error.



Source: Wikipedia, "Nyquist rate".

- ▶ **Aliasing:** If the sampling rate is too low, higher frequencies are misrepresented as lower frequencies, causing distortion.

Digitizing speech: sampling, quantization, bitrate (cont.)



Source: M. Waldrep (Dr. AIX), "Aliasing and Foldover: A Primer," Real HD-Audio, 2014.

- ▶ **Bit Depth:** Common depths include 8-bit (256 levels), 16-bit (65,536 levels), and 24-bit (16.7 million levels). Higher bit depth allows for more precise representation of amplitude.
- ▶ **Bitrate:** The amount of data processed per second, calculated as:

$$\text{Bitrate} = \text{Sampling Rate} \times \text{Bit Depth} \times \text{Number of Channels}$$

For example, a stereo audio file with a sampling rate of 44.1kHz and a bit depth of 16-bit has a bitrate of:

$$44,100 \times 16 \times 2 = 1,411,200 \text{ bits per second} = 1.4112 \text{ Mbps}$$

Pre-processing Pipeline

1. Pre-emphasis Filter

- ▶ Enhances high-frequency components of the audio signal.
- ▶ Equation:

$$y[n] = x[n] - \alpha x[n - 1]$$

where $x[n]$ is the input signal, $y[n]$ is the output, and α is typically between 0.95 and 0.97.

- ▶ Helps balance the frequency spectrum and improves feature extraction.

2. Framing

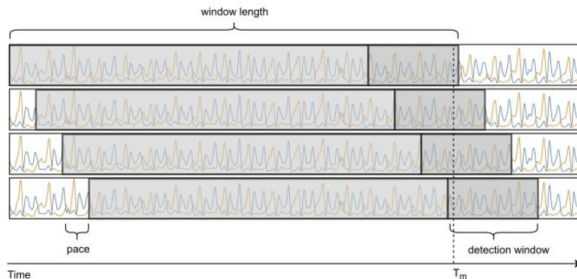
- ▶ Audio signals are divided into short frames (e.g., 25 ms) to capture local temporal features.
- ▶ Each frame overlaps with the next (commonly a 10 ms hop, i.e. 15 ms overlap).

3. Windowing

- ▶ Each frame is multiplied by a window function (commonly Hamming window) to reduce spectral leakage.
- ▶ Hamming window equation:

$$w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right)$$

where N is the frame length.



The sliding-window scheme in general form: a fixed window length advanced by a fixed step. Framing an audio signal is the same construction, with a 25 ms window and a 10 ms hop.

Source: C. Morbidoni, A. Cucchiarelli, V. Agostini, M. Knaflitz, S. Fioretti and F. Di Nardo, "Machine-Learning-Based Prediction of Gait Events From EMG in Cerebral Palsy Children," *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 29, pp. 819–830, 2021, Fig. 2 (CC BY 4.0). The traces are EMG, not speech.

Summary of Pre-processing Steps

1. **Pre-emphasis:** Boosts high frequencies.
2. **Framing:** Splits signal into short, overlapping segments.
3. **Windowing:** Applies Hamming window to each frame.

STFT & Spectrograms

- **Discrete Fourier Transform (DFT):** A mathematical operation that converts a signal from the time domain to the frequency domain. It transforms N samples of a discrete signal, $x[n]$, into N frequency bins, $X[k]$. The formula for the DFT is:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$

where $x[n]$ is the signal in the time domain, $X[k]$ is the signal in the frequency domain, N is the total number of samples, n is the sample index, and k is the frequency bin index.

- **Fast Fourier Transform (FFT):** This is not a different transform, but rather a computationally efficient algorithm for calculating the DFT.

To apply the Fourier transform in practice, you typically:

1. **Choose a window length (N):** This determines the number of samples to be transformed at a time.
2. **Choose a hop size:** This is the number of samples to advance before taking the next window.

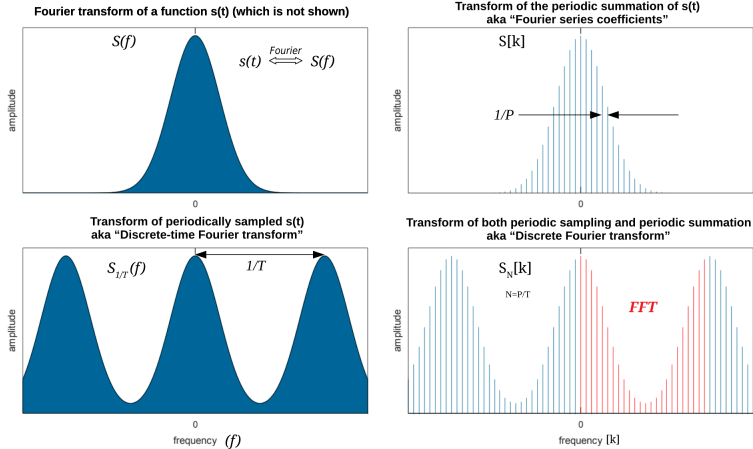
This process involves a fundamental trade-off: a longer window (N) provides better **frequency resolution** (ability to distinguish between close frequencies) but worse **time resolution** (ability to pinpoint when an event occurred).

Conversely, a shorter window improves time resolution at the expense of frequency resolution.

We use Fourier transforms to analyze the frequency content of signals. The resulting frequency information (spectra) is essential for:

- ▶ **Building Spectrograms:** Visual representations of how the frequency content of a signal changes over time.
- ▶ **Extracting Features:** Deriving key characteristics from a signal that can be used for tasks like speech recognition, music information retrieval, and audio classification.

Fourier Transforms You'll Actually Use (cont.)



Depiction of a Fourier transform. Source: Wikipedia

- ▶ STFT is a method to analyze the frequency content of a signal over time.
- ▶ It involves applying the Fourier Transform to short overlapping segments (frames) of the signal.
- ▶ The STFT provides a time-frequency representation of the signal.

► What:

- Slide a window across the signal.
- Compute a DFT per frame.

► Continuous form:

$$X(t, \omega) = \int x(\tau) w(\tau - t) e^{-j\omega\tau} d\tau$$

► Discrete idea:

- DFT of windowed frame at each time step.

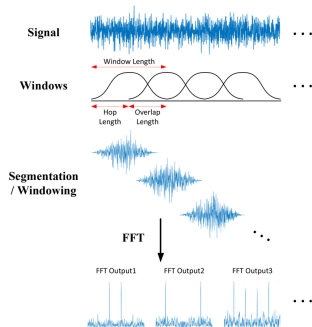
► How:

- Choose window: Hann / Hamming
- Frame length: e.g., 25 ms
- Hop: e.g., 10 ms

► Why:

- Speech is quasi-stationary in short frames.
- STFT captures how spectrum changes over time.

Short-Time Fourier Transform (STFT) (cont.)



STFT process: Signal divided into frames, each transformed into frequency domain.

Source: H. Jeon, Y. Jung, S. Lee and Y. Jung, "Area-Efficient Short-Time Fourier Transform Processor for Time-Frequency Analysis of Non-Stationary Signals," *Applied Sciences* 10(20):7208, 2020, Fig. 1 (CC BY 4.0).

- ▶ Written out, the STFT is a function of *two* variables: frame position n and frequency f ,

$$X(n, f) = \sum_{m=-\infty}^{\infty} x[m] w[m - n] e^{-j2\pi fm}$$

where $w[m - n]$ is the analysis window centred on frame n , and f is in cycles per sample.

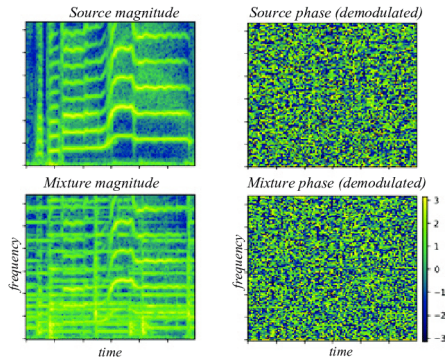
- ▶ What is actually computed is a framed DFT: with frame length N and hop H ,

$$X[n, k] = \sum_{m=0}^{N-1} x[m + nH] w[m] e^{-j2\pi km/N}, \quad k = 0, \dots, N-1$$

so the output is indexed by frame n and frequency bin k .

- ▶ The result is a complex-valued matrix, where each column represents the frequency content of a frame.
- ▶ The magnitude of the STFT gives the amplitude of each frequency at each time step.
- ▶ The phase of the STFT provides information about the timing of the frequency components.

Short-Time Fourier Transform (STFT) (cont.)



Magnitude vs. phase for a clean source (top) and a mixture (bottom). Harmonic and formant structure is visible in the magnitude; the demodulated phase looks close to noise, which is why most pipelines feed magnitude only.

Source: N. Takahashi, P. Agrawal, N. Goswami and Y. Mitsufuji, "PhaseNet: Discretized Phase Modeling with Deep Neural Networks for Audio Source Separation," Interspeech 2018, Fig. 1.

- ▶ A spectrogram is a visual representation of the STFT.
- ▶ It displays time on the x-axis, frequency on the y-axis, and color intensity represents amplitude.
- ▶ Spectrograms are widely used in audio analysis, speech recognition, and music processing.

► What:

- Heatmap of energy vs. time (x-axis) and frequency (y-axis).
- Color represents magnitude (often in dB).

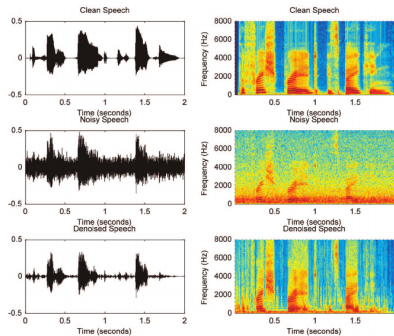
► How:

- Magnitude of STFT per time–frequency bin.
- Often log-scaled:

$$20 \log_{10} |X|$$

► Why:

- Visual patterns (formant tracks, harmonics, fricatives) → model-friendly features.
- Example: vowel formants appear as bright horizontal bands.

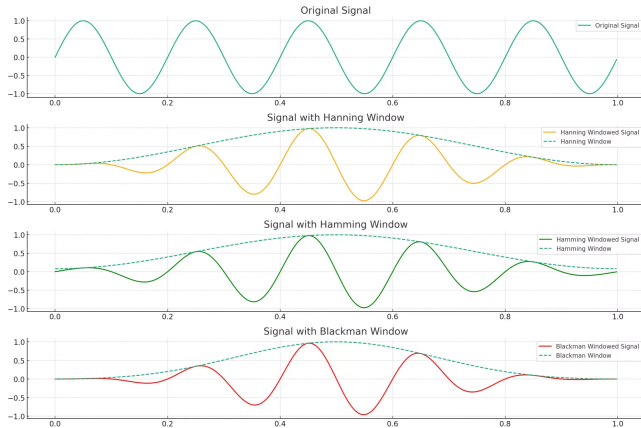


Example of a spectrogram: waveform (left) and time–frequency representation (right) of the same utterance, clean, noisy and denoised.

Source: H. Ye, G. Deng, S. J. Mauger, A. A. Hersbach, P. W. Dawson and J. M. Heasman, “A Wavelet-Based Noise Reduction Algorithm and Its Clinical Evaluation in Cochlear Implants,” *PLOS ONE* 8(9):e75662, 2013, Fig. 7 (CC BY).

- ▶ Spectrograms can be computed using various window functions (e.g., Hamming, Hanning) and different frame sizes.
- ▶ They provide insights into how the frequency content of a signal changes over time.

Spectrograms (cont.)



The same signal under a Hanning, Hamming, and Blackman window (dashed): each tapers the frame edges to zero, trading main-lobe width against side-lobe leakage.

Mel-Scale & Filterbanks

► What:

- Human pitch perception is **non-linear**.
- We perceive changes at low frequencies more finely than at high frequencies.
- Example: Difference between 200 Hz and 300 Hz feels larger than between 3000 Hz and 3100 Hz.

► How:

- Map raw frequency in Hz onto perceptual scales:
 - **Mel scale** – approximates human pitch perception.
 - **Bark scale** – based on critical bands of hearing.
 - **ERB scale** – Equivalent Rectangular Bandwidth, linked to auditory filter widths.
- Each scale compresses high-frequency regions while preserving low-frequency resolution.

► Why:

- Features reflecting human hearing are more effective for **Automatic Speech Recognition (ASR)** and **Text-to-Speech (TTS)**.
- Speech features (e.g., MFCCs) rely heavily on perceptual scales.

► Focus: Mel Scale

- Most widely used in speech/audio tasks.
- Roughly linear spacing below 1 kHz, logarithmic above.

► What:

- Mel scale converts frequency f (Hz) into perceptual pitch units (mels).
- Forward transform:

$$m = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$$

- Inverse transform:

$$f = 700 \left(10^{\frac{m}{2595}} - 1 \right)$$

- The constant 2595 is not arbitrary: it is chosen so that 1000 Hz lands on 1000 mels.

► How:

- Use Mel spacing to set filterbank center frequencies.
- Commonly used in **Mel Filterbanks** and **MFCCs**.

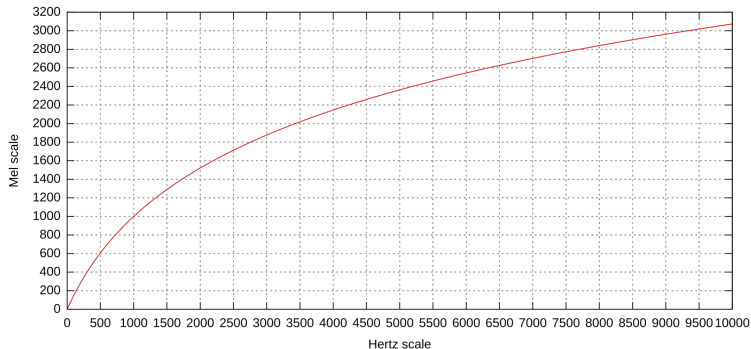
► Why:

- Aligns frequency resolution with **human pitch perception**.
- Compresses high-frequency regions (less perceptually important).
- Reduces unhelpful detail while retaining speech-relevant cues.

► Example:

- By convention, $1000 \text{ Hz} \approx 1000 \text{ mels}$.
- Below 1 kHz: roughly linear relation.
- Above 1 kHz: increasingly logarithmic.

Mel Scale: Definition & Intuition (cont.)

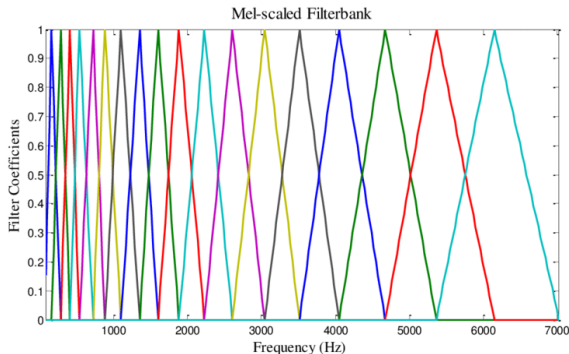


Plots of pitch mel scale versus hertz scale.

Source: Wikipedia, "Mel scale" (Wikimedia Commons, File:Mel-Hz plot.svg, by Krishna Vedala, CC BY-SA 3.0).

Mel Filterbanks:

- ▶ The frequency axis is warped to the Mel scale.
- ▶ Typically, N triangular filters (e.g., $N = 40$) are created, spaced evenly in the Mel domain.
- ▶ Each filter collects energy from a range of frequencies, emphasizing perceptually important bands.
- ▶ The output is a set of Mel filterbank energies, which are used as features for audio and speech processing tasks.



Mel filterbank basis functions, here using 20 Mel-filters over 0–7000 Hz.

Source: Yusnita M. A., Paulraj M. P., S. Yaacob, R. Yusuf and Shahrman A. B., "Analysis of Accent-Sensitive Words in Multi-Resolution Mel-Frequency Cepstral Coefficients for Classification of Accents in Malaysian English," *International Journal of Automotive and Mechanical Engineering*, vol. 7, pp. 1053–1073, 2013, Fig. 3.

Filterbank Construction Steps:

1. Convert the lower and upper frequency bounds to Mel scale.
2. Linearly space $N + 2$ points between these bounds in Mel scale.
3. Convert these points back to Hz to get filter edges.
4. For each filter k , define a triangular response:

$$H_k(f) = \begin{cases} 0, & f < f_{k-1} \\ \frac{f - f_{k-1}}{f_k - f_{k-1}}, & f_{k-1} \leq f < f_k \\ \frac{f_{k+1} - f}{f_{k+1} - f_k}, & f_k \leq f < f_{k+1} \\ 0, & f \geq f_{k+1} \end{cases} \quad (1)$$

5. Multiply the power spectrum by each filter and sum to get filterbank energies.

- ▶ Different feature types capture sound information at different levels:
 - **Spectrogram (linear-Hz):** closest to the raw physics of sound; preserves fine high-frequency detail.
 - **Mel-spectrogram:** compresses the high-frequency range using perceptual spacing, better reflecting human hearing.
 - **Log-Mel:** adds logarithmic compression to mimic loudness perception and control dynamic range.
 - **MFCCs (Mel-Frequency Cepstral Coefficients):** apply a DCT to Log-Mel features, creating compact, decorrelated coefficients widely used in classic ASR.

► Common processing pipeline:

Waveform $\rightarrow$ STFT $\rightarrow$ Mel scaling (optional) $\rightarrow$ log $\rightarrow$ DCT (optional)

► Why this matters:

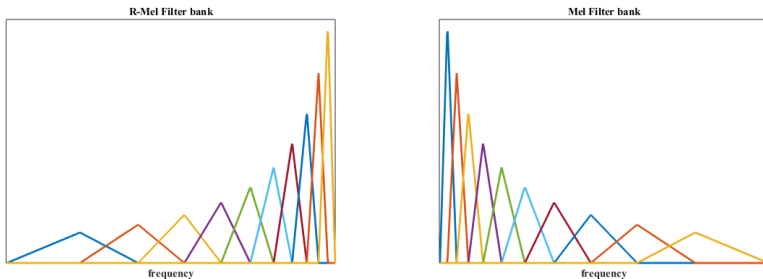
- More compact and stable features are easier for machine learning models to work with.
- Traditional models (like GMM-HMMs, SVMs) rely heavily on these, but even neural networks benefit when input dimensionality is reduced.

- ▶ A mel-spectrogram is a **time** $\times$ **mel-band matrix** created by re-binning STFT magnitudes through triangular filters on the Mel scale.
- ▶ The process:
 - Start with STFT $\rightarrow$ compute the power spectrum.
 - Apply a mel filterbank (typically 40–80 bands).
 - Take logarithms of the magnitudes to compress the dynamic range.
- ▶ Intuition:
 - Compact representation, aligned with human pitch perception.
 - Retains enough structure to capture speech/music content.
 - Works especially well with **CNNs** and **TTS acoustic models**.

- Equation (conceptual form):

$$E_m = \sum_k |X[k]|^2 H_m[k]$$

where H_m is the m -th triangular mel filter applied to the power spectrum.



Reversed-mel (left) and mel (right) filter banks.

Source: Wikimedia Commons, File:Mel RMel.png, by Ghasemzadehh, CC BY-SA 4.0.

- ▶ Log-Mel features are created by taking the logarithm of mel-band energies:

$$\log(E_m)$$

where E_m is the energy in the m -th mel band.

- ▶ Processing pipeline:

- Compute STFT $\rightarrow$ apply mel filterbank $\rightarrow$ mel-band energies.
- Apply logarithmic compression, either:

$$\log(E_m) \quad \text{or} \quad \log(1 + \alpha E_m)$$

to avoid issues with very small values.

► Why this step matters:

- Human perception of loudness is approximately logarithmic.
- Log scaling stabilizes feature magnitudes, preventing loud frames from dominating.
- Improves optimization for downstream machine learning models.

► Applications:

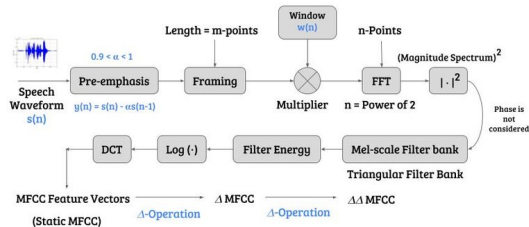
- Ubiquitous in modern **TTS systems**: log-mel spectrograms are the interface between the acoustic model and the vocoder (e.g., Tacotron 2 predicts log-mel frames, WaveNet turns them into a waveform).
- Widely used in speech recognition pipelines: Whisper takes 80-channel log-mel input.

- ▶ Mel-Frequency Cepstral Coefficients (MFCCs) provide a compact summary of the log-Mel spectrum, obtained by decorrelating features with a Discrete Cosine Transform (DCT).
- ▶ This representation captures the **spectral envelope** (linked to vocal-tract shape) while discarding fine pitch details.
- ▶ Typical processing pipeline:
 - (Optional) pre-emphasis to boost high frequencies.
 - Frame the signal, apply a window, and compute the STFT.
 - Apply a mel filterbank to obtain band energies.
 - Take logarithms of the energies.
 - Apply DCT to decorrelate; keep the first Q coefficients (e.g., 13).
 - Optionally append temporal derivatives (Δ) to capture dynamics.

► Why MFCCs are important:

- Historically very strong for HMM-GMM based ASR systems.
- Emphasize formant structure and vocal-tract characteristics.
- Tend to suppress pitch information, which is often less useful for recognition tasks.

- ▶ MFCCs are widely used in speech and audio processing.
- ▶ They capture the timbral texture of audio signals.
- ▶ Typically, only the first 13 coefficients are used for feature extraction.



A block diagram for MFCC computation: From audio signal to MFCC features.

Source: B. Barai, D. Das, N. Das, S. Basu and M. Nasipuri, "VQ/GMM-Based Speaker Identification with Emphasis on Language Dependency," *Advances in Intelligent Systems and Computing* 810, Springer, 2019, Fig. 2.

Pipeline:

1. Compute log-Mel spectrogram
2. Apply Discrete Cosine Transform (DCT)
3. Keep first 13 coefficients (MFCCs)

MFCC Equation:

$$c_k = \sum_{n=1}^N \log(E_n) \cos \left[\frac{\pi k}{N} \left(n - \frac{1}{2} \right) \right] \quad (2)$$

where:

- ▶ c_k is the k -th MFCC coefficient
- ▶ E_n is the energy in the n -th Mel filter
- ▶ N is the total number of Mel filters

- MFCCs are built from three main mathematical steps:

1. **Band energies (Mel filtering):**

$$E_m = \sum_k |X[k]|^2 H_m[k]$$

where H_m is the m -th triangular Mel filter.

2. **Logarithmic compression:**

$$L_m = \log(E_m + \epsilon)$$

which stabilizes values and mimics loudness perception.

3. **DCT-II (decorrelation):**

$$c_q = \sum_{m=1}^M L_m \cos \left[\frac{\pi q}{M} \left(m - \frac{1}{2} \right) \right], \quad q = 0, \dots, Q - 1$$

where M is the number of Mel bands and Q the number of coefficients (often $Q = 13$). The c_0 coefficient is sometimes dropped if energy is handled separately.

► Practical notes:

- Libraries like `librosa` or `torchaudio` implement this pipeline directly.
- Typical settings: $M = 40-80$ Mel bands, $Q \approx 13$ coefficients.

► Why this works:

- The DCT decorrelates the log-Mel energies.
- Produces compact, roughly uncorrelated features suited for traditional models.
- Historically central to HMM-GMM ASR pipelines.

Linear Predictive Coding (LPC):

- ▶ Models the speech signal as a linear combination of past samples.
- ▶ Useful for speech compression and speaker recognition.
- ▶ Captures the spectral envelope efficiently.

Perceptual Linear Prediction (PLP):

- ▶ Incorporates psychoacoustic models to mimic human hearing.
- ▶ Reduces spectral information to perceptually relevant features.
- ▶ Often used in robust speech recognition systems.

Pitch:

- ▶ Represents the fundamental frequency of speech.
- ▶ Important for prosody analysis, emotion detection, and speaker identification.

- ▶ Extracted using autocorrelation or cepstral methods.

Formants:

- ▶ Resonant frequencies of the vocal tract.
- ▶ Key for phoneme classification and speech synthesis.
- ▶ Estimated using LPC or spectral analysis.

Spectrogram-based Learned Features:

- ▶ Deep learning models (CNNs, RNNs) extract features directly from spectrograms.
- ▶ Capture complex patterns and temporal dependencies.
- ▶ Widely used in modern end-to-end speech and audio systems.

When to Choose What:

- ▶ **LPC/PLP:** When computational efficiency and interpretability are important; suitable for traditional speech tasks.
- ▶ **Pitch/Formants:** For tasks involving prosody, emotion, or phoneme-level analysis.
- ▶ **Spectrogram-based Learned Features:** For large-scale, data-driven applications where a deep model has enough data to learn its own filters from the spectrogram.
- ▶ Consider the task requirements, available data, and computational resources when selecting features.

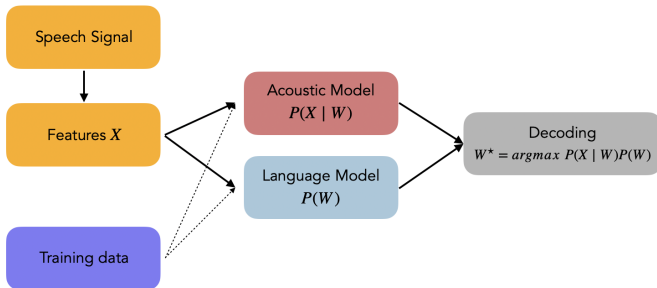
Automatic Speech Recognition (ASR)

- ▶ Automatic Speech Recognition converts spoken language (audio) into written text.
- ▶ Example:

Audio: “Hey computer, what’s the weather?” → Text: “hey computer what’s the weather”

- ▶ How it works (high-level pipeline):
 - Extract features from audio (e.g., log-Mel spectrograms, MFCCs).
 - Pass features through an **acoustic model** to map audio patterns to phonemes or subword units.
 - Use a **language model** to predict the most probable word sequences.
 - Decode the combined outputs into final text.

- ▶ Why ASR is important:
 - Enables natural language interaction with computers (voice assistants).
 - Facilitates transcription, accessibility, and spoken-input NLP applications.



Statistical historical approach to ASR.

Source: M. Fabien, "Introduction to Automatic Speech Recognition (ASR),"

https://maelfabien.github.io/machinelearning/speech_reco/

- ▶ Traditional ASR systems were built as modular pipelines with interpretable components.
- ▶ Main components:
 - **Acoustic model:** estimates $P(x \mid \text{phones})$, mapping audio features to phonetic units.
 - **Pronunciation dictionary:** converts phones to words.
 - **Language model:** estimates $P(\text{words})$, guiding likely word sequences.
 - **Decoder:** searches for the most probable word sequence given the acoustic and language models.
- ▶ Mathematical formulation:

$$\hat{W} = \arg \max_W P(W \mid X) = \arg \max_W P(X \mid W) P(W)$$

where W = word sequence, X = extracted audio features.

- ▶ Why this approach worked:

- Easier to engineer and understand.
- Interpretable: each module can be improved independently.

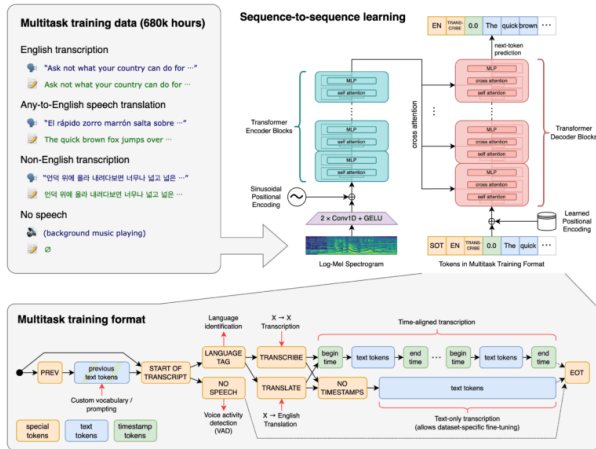
► Limitations:

- Complex, hand-crafted system.
- Errors can propagate through the pipeline.

- ▶ Modern ASR systems often replace traditional modular pipelines with **neural sequence-to-sequence models**, directly mapping audio features to text.
- ▶ Popular approaches:
 - **CTC (Connectionist Temporal Classification)** – handles variable-length alignments.
 - **Attention-based encoder-decoder** – learns soft alignments between input frames and output tokens.
 - **Transducer models** – integrate CTC-style alignment with recurrent decoding.
- ▶ Advantages:
 - Minimal handcrafting of features or components.
 - Often achieves better accuracy when trained on large datasets.
- ▶ Each approach solves the same problem, that the audio has far more frames than the transcript has tokens, in a different way. We take them one at a time in the sections that follow.

- ▶ Reference: Hannun, “Sequence Modeling With CTC”, Distill, 2017.
<https://distill.pub/2017/ctc/>

- ▶ Whisper is a state-of-the-art, multilingual ASR model developed by OpenAI.
- ▶ Key architecture:
 - Encoder-decoder transformer.
 - Trained on 680,000 hours of **weakly supervised** audio: transcripts scraped from the web and paired with audio, not annotated for the task. 117,000 of those hours are non-English.
 - Takes 80-channel log-Mel spectrograms as input (25 ms window, 10 ms hop).
- ▶ Advantages:
 - Robust to different accents and noisy environments.
 - Supports multitask learning: ASR and optional translation.
- ▶ Example use case:
 - Spoken French: “Je parle français” → can output English translation.



Source: Radford et al., "Robust Speech Recognition via Large-Scale Weak Supervision", 2022, Fig. 1.

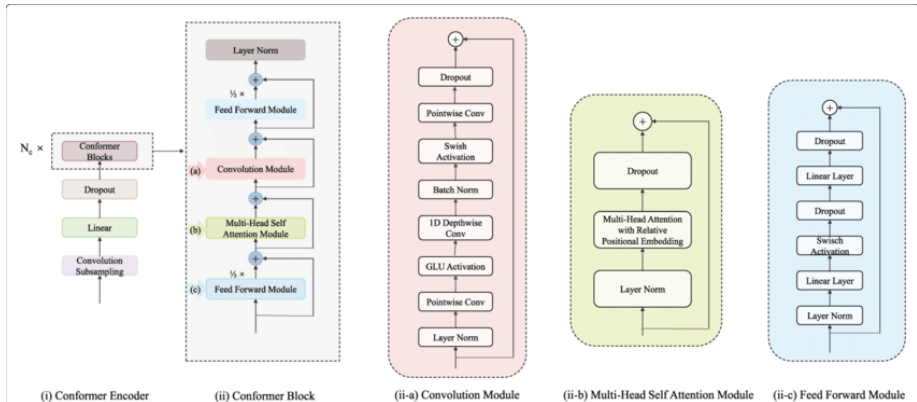
- ▶ Conformer is a hybrid neural model that combines **convolutions** and **transformers** to process speech effectively.
- ▶ Core components:
 - **Convolution:** captures local structures and fine-grained temporal patterns in audio.
 - **Self-attention:** captures global context and long-range dependencies.
- ▶ Achievements:
 - Was state of the art on LibriSpeech when published: 2.1% / 4.3% WER on test-clean / test-other without a language model, and 1.9% / 3.9% with an external LM (Conformer (L), 118.8M parameters).
 - Balances efficiency and accuracy better than pure transformer or CNN models alone: a 10.3M-parameter Conformer (S) still reaches 2.7% / 6.3%.

- Block equations: the four modules are applied *in sequence*, each as a pre-norm residual, with the two feed-forward modules half-weighted and sandwiching the attention and convolution pair:

$$\begin{aligned}\tilde{x} &= x + \frac{1}{2}\text{FFN}(x) & x' &= \tilde{x} + \text{MHSA}(\tilde{x}) \\ x'' &= x' + \text{Conv}(x') & y &= \text{LayerNorm}(x'' + \frac{1}{2}\text{FFN}(x''))\end{aligned}$$

where MHSA = multi-head self-attention (with relative positional encoding), Conv = convolutional module, FFN = feed-forward network.

Conformer (Conv + Transformer) (cont.)



Conformer encoder and its sub-modules, after Gulati et al., "Conformer", Interspeech 2020, Fig. 1. Redrawn version from Li et al., "Adversarial joint training with self-attention mechanism for robust end-to-end speech recognition", EURASIP J. Audio Speech Music Process., 2021.

- ▶ Whisper and Conformer represent two modern approaches to ASR with complementary strengths:
- ▶ **Whisper:**
 - Multilingual, trained on a massive weakly supervised dataset.
 - Robust to accents, noise, and varied conditions.
 - Open-sourced, general-purpose.
- ▶ **Conformer:**
 - Efficient hybrid model combining convolution + transformer.
 - Excels on English benchmarks like LibriSpeech.
 - Often fine-tuned for specific tasks.

► Trade-offs:

- Whisper is more general-purpose, handles diverse languages and noisy conditions.
- Conformer achieves very low WER on read English speech (2.1% on LibriSpeech test-clean without an LM) but may require task-specific tuning.

- ▶ Understanding the types of errors in ASR helps diagnose model weaknesses and improve performance.
- ▶ Common error types:
 - **Substitution:** a word is replaced by another. Example: “cat” → “cap”
 - **Deletion:** a word is dropped. Example: “the cat sat” → “cat sat”
 - **Insertion:** an extra word is added. Example: “cat sat” → “the cat sat”

- ▶ Several factors influence how well an ASR system performs in real-world conditions:
- ▶ **Noise level and microphone quality:** Background noise or poor recording devices can degrade feature extraction and recognition accuracy.
- ▶ **Speaker variability:** Differences in accent, pitch, speaking rate, or pronunciation can challenge the acoustic model.
- ▶ **Domain mismatch:** Models trained on casual speech may struggle with specialized domains (e.g., medical or legal speech).
- ▶ **Training data size and diversity:** More diverse and larger datasets generally improve robustness across speakers, accents, and environments.

CTC: Solving the Alignment Problem in Sequence-to-Sequence Tasks

Many sequence tasks (e.g., speech-to-text) face the **alignment problem**: input and output sequences are of different lengths and not aligned. CTC enables training without explicit alignment.

- ▶ **Input:** Sequence $\mathbf{x} = (x_1, x_2, \dots, x_T)$
- ▶ **Output:** Sequence $\mathbf{y} = (y_1, y_2, \dots, y_U)$
- ▶ **Challenge:** $T \neq U$, unknown alignment

CTC introduces a special blank token ($\emptyset$) and allows repeated labels. The final output is obtained by collapsing repeats and removing blanks. Note what the blank buys us: without it, a genuine double letter such as the “ll” in *hello* would collapse to a single “l”. A blank between the two repeats keeps them apart.

CTC: Training Without Alignments (cont.)



CTC: Blank token insertion and collapsing (the blank is written ϵ here).

Source: A. Hannun, "Sequence Modeling with CTC," *Distill*, 2017. <https://distill.pub/2017/ctc>

CTC Loss Function:

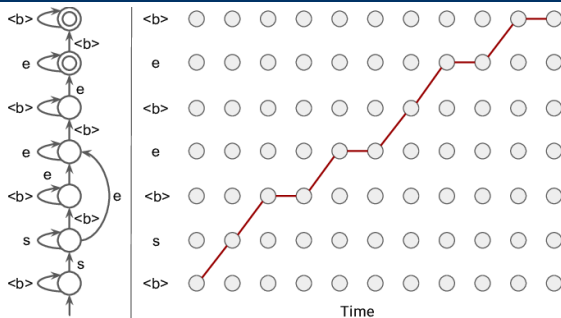
$$L_{CTC} = -\ln \left(\sum_{\pi \in \mathcal{B}^{-1}(\mathbf{y})} \prod_{t=1}^T p(\pi_t | \mathbf{x}) \right) \quad (3)$$

- ▶ π : a possible alignment (path) with blanks and repeats
- ▶ $\mathcal{B}^{-1}(\mathbf{y})$: set of all paths that collapse to $\mathbf{y}$
- ▶ $p(\pi_t | \mathbf{x})$: probability of label π_t at time t

CTC Decoding:

1. Predict a sequence of labels (including blanks) for each time step.
2. Collapse repeated labels and remove blanks to get the final output.

CTC: Training Without Alignments (cont.)



CTC alignment for the target "see": the automaton of all valid paths (left) and one alignment through the lattice (right); the blank is written .

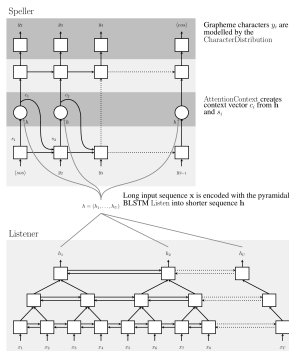
Source: R. Prabhavalkar, T. Hori, T. N. Sainath, R. Schlüter and S. Watanabe, "End-to-End Speech Recognition: A Survey," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 32, pp. 325–351, 2024, Fig. 1.

Encoder–Decoder Overview:

- ▶ **Encoder:** Processes the input acoustic features (e.g., Mel-spectrograms) and encodes them into a sequence of hidden representations.
- ▶ **Decoder:** Generates the output transcription, one token at a time, conditioned on the encoder outputs and previous decoder states.
- ▶ **Attention Mechanism:** Allows the decoder to focus on relevant parts of the input sequence at each decoding step, improving alignment and performance.
- ▶ Unlike CTC, nothing forces the alignment to be monotonic and no independence assumption is made across output tokens: the decoder is free to attend anywhere.

Listen, Attend and Spell (LAS): the canonical instance of this design.

- ▶ The **listener** is a pyramidal BLSTM encoder that shortens the frame sequence $\mathbf{x}$ (length T) into $\mathbf{h}$ (length $U \leq T$).
- ▶ The **speller** is an attention-based decoder emitting characters y_i one at a time.



Source: Chan et al., "Listen, Attend and Spell", ICASSP 2016, Fig. 1.

Attention Alignment:

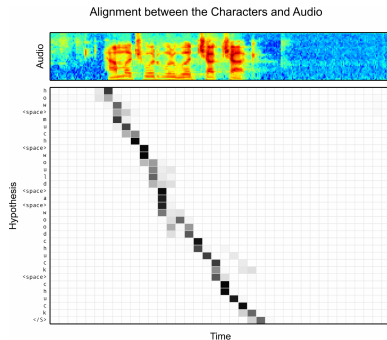
At each decoder time step t , the attention mechanism computes a context vector as a weighted sum of encoder outputs. The weights $\alpha_{t,s}$ represent the alignment between decoder step t and encoder position s :

$$\alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{s'} \exp(e_{t,s'})} \quad (4)$$

where $e_{t,s}$ is the attention score (e.g., computed via a feedforward network) between decoder state at time t and encoder output at position s .

What the alignment looks like in practice

- ▶ Plotting $\alpha_{t,s}$ for every output character gives a heatmap of decoder step against audio frame.
- ▶ For speech it comes out close to diagonal, because speech is produced in order, even though nothing in the model enforces that.
- ▶ Where the model is unsure the mass spreads out: below, “woodchuck” draws a diluted band because “woodchuck” and “chuck” sound alike.

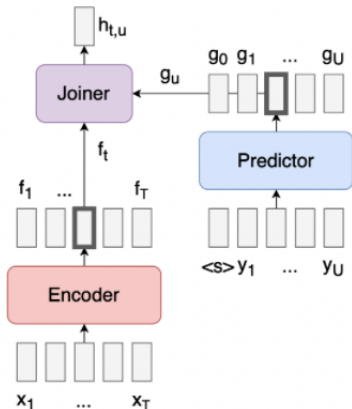


Alignment for the utterance “how much would a woodchuck chuck”. Source: Chan et al., “Listen, Attend and Spell”, ICASSP 2016, Fig. 2.

RNN-Transducer (RNN-T) is a sequence-to-sequence model widely used for end-to-end speech recognition. It extends the CTC (Connectionist Temporal Classification) approach by modeling both input and output dependencies:

- ▶ **Sequence-to-sequence model:** Maps input sequences (e.g., acoustic features) directly to output sequences (e.g., transcriptions).
- ▶ **End-to-end speech recognition:** Eliminates the need for separate components (like acoustic, pronunciation, and language models).
- ▶ **Extension of CTC:**
 - CTC models only input dependencies and assumes output tokens are conditionally independent.
 - RNN-T models both input and output dependencies, allowing the prediction of each output token to depend on previous outputs.
- ▶ **Key advantage:** Enables more accurate modeling of language and flexible alignments between input and output.

RNN-Transducer (RNN-T): Overview (cont.)



RNN-T architecture. The prediction network and joint network are also called the *predictor* and the *joiner*. Source: L. Lugosch, "Sequence-to-sequence learning with Transducers," 2020. <https://lorenlugosch.github.io/posts/2020/11/transducer/>

Architecture Components:

- ▶ **Encoder:** Processes the input acoustic features and produces high-level representations.
- ▶ **Prediction Network:** Acts like a language model, conditioning on previous non-blank output tokens.
- ▶ **Joint Network:** Combines encoder and prediction network outputs to produce logits over the output vocabulary (including the blank symbol).

$$\mathbf{h}_t = \text{Encoder}(\mathbf{x}_{1:t}) \quad (5)$$

$$\mathbf{g}_u = \text{PredictionNetwork}(y_{1:u-1}) \quad (6)$$

$$\mathbf{z}_{t,u} = \text{JointNetwork}(\mathbf{h}_t, \mathbf{g}_u) \quad (7)$$

Loss Overview:

- ▶ RNN-T loss marginalizes over all possible alignments between input and output sequences.
- ▶ Similar to CTC, but allows for output dependencies.
- ▶ Computed efficiently using dynamic programming.

$$\mathcal{L}_{\text{RNN-T}} = -\log \sum_{\pi \in \mathcal{A}(\mathbf{y}, T)} P(\pi | \mathbf{x}) \quad (8)$$

where $\mathcal{A}(\mathbf{y}, T)$ is the set of all valid alignments.

Pros vs. CTC:

- ▶ Models output dependencies (better language modeling).
- ▶ No need for external language model during inference.
- ▶ More flexible alignments.

Cons vs. CTC:

- ▶ More complex and computationally intensive: the loss is computed over a $T \times U$ lattice, so training memory grows with the product of input and output lengths.
- ▶ Harder to train and tune.
- ▶ Decoding is slower due to output dependencies.

Transformer & Conformer in ASR

- ▶ **Transformers** and **Conformers** are state-of-the-art architectures for Automatic Speech Recognition (ASR).
- ▶ Both use self-attention to model long-range dependencies in audio sequences.
- ▶ Conformer extends Transformer by integrating convolutional modules for local feature extraction.
- ▶ Key components include:
 - Multi-head self-attention
 - Feed-forward networks
 - Residual connections and layer normalization

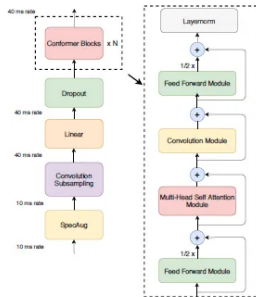


Figure 1: **Conformer encoder model architecture.** Conformer comprises of two macaron-like feed-forward layers with half-step residual connections sandwiching the multi-headed self-attention and convolution modules. This is followed by a post layernorm.

Source: Gulati et al., "Conformer", Interspeech 2020, Fig. 1.

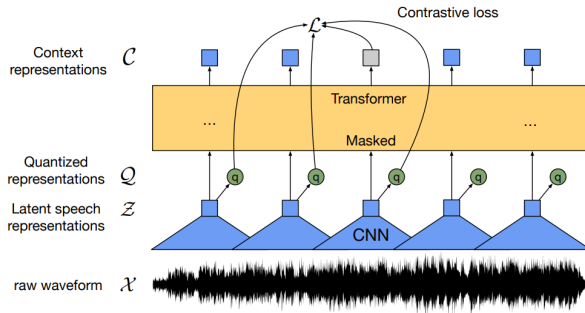
The encoder blocks are the ones covered earlier. Two things about speech force adjustments:

- ▶ **The sequence is very long.** At a 10 ms hop, one second of audio is 100 frames and a 30-second utterance is 3000, against perhaps 60 word-pieces of transcript. Self-attention costs $O(T^2)$, so ASR encoders first apply **convolutional subsampling**, typically two strided convolutions that cut the frame rate by $4\times$ (10 ms $\rightarrow$ 40 ms) before any attention layer runs.
- ▶ **Position matters relatively, not absolutely.** An utterance can start at any offset and last any length, so Conformer replaces absolute positional encoding with the **relative** sinusoidal scheme, making a block's behaviour depend on the distance between frames rather than their index.

Data augmentation on the input is also standard: **SpecAugment** masks bands of frequency and spans of time directly in the log-mel spectrogram.

- ▶ **Conformer** combines self-attention and convolution for effective speech modeling.
- ▶ **Block Structure:** two *half-step* feed-forward modules sandwich the attention and convolution pair (the “macaron” arrangement):
 - $\frac{1}{2}$ Feed-Forward $\rightarrow$ Multi-Head Self-Attention $\rightarrow$ Convolution $\rightarrow$ $\frac{1}{2}$ Feed-Forward $\rightarrow$ LayerNorm
- ▶ **Key Features:**
 - **Convolution Module:** LayerNorm $\rightarrow$ pointwise conv (expansion 2) $\rightarrow$ GLU $\rightarrow$ 1-D depthwise conv $\rightarrow$ BatchNorm $\rightarrow$ Swish $\rightarrow$ pointwise conv $\rightarrow$ dropout.
 - Every sub-module is a pre-norm residual unit; attention uses relative sinusoidal positional encoding.

- ▶ Proposed by Baevski et al. (2020): self-supervised learning for speech representation, the speech counterpart of BERT-style pretraining.
- ▶ **Architecture:** a CNN encoder turns the raw waveform into latent representations z_t ; a transformer produces context representations c_t ; a quantization module gives discrete targets q_t .
- ▶ **Pretraining:** mask spans of the latent sequence and train contrastively to pick the true quantized target among distractors, on unlabeled audio alone.



Source: Baevski et al., "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations", NeurIPS 2020, Fig. 1.

Fine-tuning for ASR with Wav2Vec 2.0

- ▶ **Self-Supervised Learning (SSL):** Pre-train Wav2Vec 2.0 on large unlabeled audio.
- ▶ **Fine-tuning:** Use small labeled dataset for ASR.
- ▶ **Loss Function:** Connectionist Temporal Classification (CTC) Loss.
- ▶ **Goal:** Map audio features to text transcriptions.

- ▶ **Small labeled data:** Only 10 minutes of transcribed speech.
- ▶ **Performance:** Word Error Rate (WER) 4.8% (test-clean) / 8.2% (test-other).
- ▶ **Read the small print.** That headline number needs all three of:
 - the `LARGE` model (~ 317 M parameters),
 - pre-training on 53k hours of unlabeled LibriVox audio (LV-60k),
 - a Transformer language model at decoding time.

Swap the LM for a 4-gram and the same model gives 6.6% / 10.3%; decode with no LM at all and it is around 40%. The `BASE` model at 10 minutes gives 6.9% / 12.9%.

- ▶ **Efficient:** High accuracy with minimal *supervision*, not with minimal compute or data.

Model	Unlabeled data	LM	dev		test	
			clean	other	clean	other
10 min labeled						
Discrete BERT [4]	LS-960	4-gram	15.7	24.1	16.3	25.2
BASE	LS-960	4-gram	8.9	15.7	9.1	15.6
		Transf.	6.6	13.2	6.9	12.9
LARGE	LS-960	Transf.	6.6	10.6	6.8	10.8
	LV-60k	Transf.	4.6	7.9	4.8	8.2
1h labeled						
Discrete BERT [4]	LS-960	4-gram	8.5	16.4	9.0	17.6
BASE	LS-960	4-gram	5.0	10.8	5.5	11.3
		Transf.	3.8	9.0	4.0	9.3
LARGE	LS-960	Transf.	3.8	7.1	3.9	7.6
	LV-60k	Transf.	2.9	5.4	2.9	5.8
10h labeled						
Discrete BERT [4]	LS-960	4-gram	5.3	13.2	5.9	14.1
Iter. pseudo-labeling [58]	LS-960	4-gram+Transf.	23.51	25.48	24.37	26.02
	LV-60k	4-gram+Transf.	17.00	19.34	18.03	19.92
BASE	LS-960	4-gram	3.8	9.1	4.3	9.5
		Transf.	2.9	7.4	3.2	7.8
LARGE	LS-960	Transf.	2.9	5.7	3.2	6.1
	LV-60k	Transf.	2.4	4.8	2.6	4.9

Source: Baevski et al., “wav2vec 2.0”, NeurIPS 2020, Table 1 (extract).

Wav2Vec 2.0 + CTC Loss (PyTorch)

```
import torch
import torchaudio
from transformers import Wav2Vec2ForCTC, Wav2Vec2Processor

processor = Wav2Vec2Processor.from_pretrained("facebook/wav2vec2-base-960h")
model = Wav2Vec2ForCTC.from_pretrained("facebook/wav2vec2-base-960h")

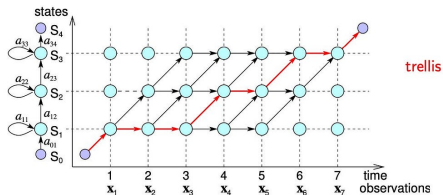
input_audio, _ = torchaudio.load("audio.wav")
inputs = processor(input_audio.squeeze(), sampling_rate=16000,
                   return_tensors="pt")
with torch.no_grad():
    logits = model(**inputs).logits
pred_ids = torch.argmax(logits, dim=-1)
transcription = processor.decode(pred_ids[0])
print(transcription)
```

► CTC Decoding:

- Greedy decoding: Select most probable token at each timestep.
- Beam search: Explore multiple hypotheses for better accuracy.
- Viterbi algorithm: Find most likely sequence.

► Equation:

$$\hat{y} = \arg \max_y P(y|x)$$



$$p(\mathbf{X}, \text{path}_\ell | \lambda) = p(\mathbf{X} | \text{path}_\ell, \lambda) P(\text{path}_\ell | \lambda)$$

$$\text{likelihood: } \sum_{\{\text{path}_\ell\}} p(\mathbf{X}, \text{path}_\ell | \lambda)$$

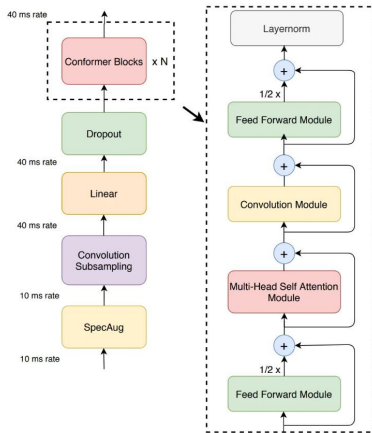
$$\text{decode: } \max_{\text{path}_\ell} p(\mathbf{X}, \text{path}_\ell | \lambda)$$

The same sum-over-paths (likelihood) vs. max-over-paths (Viterbi) choice, drawn on an HMM trellis.

Source: H. Shimodaira and S. Renals, "Hidden Markov Models and Gaussian Mixture Models," Automatic Speech Recognition Lectures 4–5, University of Edinburgh.

Conformer-CTC: Non-autoregressive ASR

- **Conformer:** Combines convolutional and transformer layers.
- **CTC Loss:** Connectionist Temporal Classification for sequence alignment.
- **Non-autoregressive:** Processes entire input sequence in parallel.



Source: Gulati et al., "Conformer," Interspeech 2020,

Fig. 1.

► CTC (Connectionist Temporal Classification):

- Aligns input and output sequences without explicit segmentation.
- Every output token is predicted independently, so all frames decode in parallel.
- Simpler and faster inference; benefits most from an external language model.

► Transducer Models:

- Jointly model alignment and output prediction.
- Autoregressive in the output: each token conditions on the previous ones.
- Better for streaming and online decoding, since the encoder need not see the whole utterance.
- More complex, higher accuracy in some cases.

- ▶ **NVIDIA Riva / NeMo:** production ASR toolkits shipping pre-trained Conformer-CTC checkpoints, optimized for GPU inference and both real-time and batch processing.
- ▶ **Parakeet:** a family of NeMo models built on a **FastConformer** encoder, a redesigned Conformer with a more aggressive downsampling schema that runs about $2.8\times$ faster, paired with a CTC, RNN-T, or TDT (token-and-duration transducer) decoder.
- ▶ **Citrinet:** a fully convolutional non-autoregressive CTC model, 1-D time-channel separable convolutions plus squeeze-and-excitation, from 9.8M to 142M parameters. Fast because CTC decoding is non-autoregressive; Riva now marks it superseded by Parakeet and Conformer-CTC.
- ▶ **References:**
 - [Conformer \(arXiv:2005.08100\)](#), [FastConformer \(arXiv:2305.05084\)](#), [Citrinet \(arXiv:2104.01721\)](#)
 - [NVIDIA Riva ASR model docs](#)

- ▶ **SpeechBrain:** open-source toolkit for speech processing.
- ▶ Supports streaming ASR through a Conformer *transducer* trained with dynamic chunk training, so the encoder only ever sees a bounded chunk of future audio. CTC is kept as an auxiliary loss to stabilise training.
- ▶ Released streaming checkpoints for LibriSpeech and GigaSpeech.



The three output shapes SpeechBrain recipes cover: a class label, a continuous value, and a transcript. Source: SpeechBrain (Ravanelli et al., 2021).

- ▶ Conformer-CTC enables fast, parallel ASR.
- ▶ CTC models are simpler and efficient, at the cost of assuming output tokens are independent.
- ▶ NVIDIA NeMo/Riva and SpeechBrain provide production implementations.
- ▶ Streaming needs bounded lookahead, which is why streaming recipes usually reach for a transducer rather than plain CTC.

Spoken Language Understanding (SLU)

- ▶ SLU focuses on understanding the meaning of spoken utterances, going beyond mere transcription.
- ▶ Goal: transform “what was said” → “what was meant.”
- ▶ Input and output:
 - Input: audio signal.
 - Output: structured meaning, typically **intent** + **slots**.
- ▶ Definitions:
 - **Intent**: the action the user wants to perform (e.g., play music, set an alarm).
 - **Slots**: arguments of the intent (e.g., {song: “Yesterday”, artist: “Beatles”}).

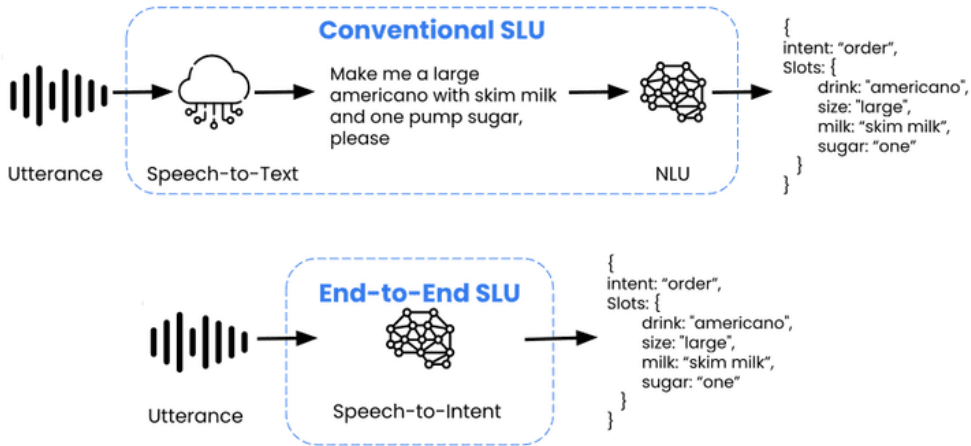
► Example:

- Audio: "Set an alarm for 7 a.m."
- Output: {intent: set_alarm, time: 7:00}

► Why SLU is important:

- Enables interaction with smart assistants and dialogue systems.
- Bridges the gap between raw speech signals and actionable NLP tasks.

Spoken Language Understanding (SLU) (cont.)



Spoken Language Understanding. Source: [PICOVOICE.AI](https://picovoice.ai)

- ▶ Many commercial SLU systems follow a two-step pipeline, separating speech recognition from understanding.
- ▶ Pipeline steps:
 1. **Audio → ASR → Text Transcript** Converts raw audio into written words.
 2. **Text → NLU Model**
 - ▶ **Intent classification:** identifies user's action (e.g., `request_weather`).
 - ▶ **Slot filling:** extracts entities or arguments (e.g., {location: "London"}).
 3. **Dialogue manager:** decides system response or action based on intent and slots.

► Advantages of this approach:

- Reuses existing NLP models trained on text.
- Easier to build and maintain using large text corpora.

► Example:

- User says: "What's the weather like in Paris?"
- ASR output: "what's the weather like in paris"
- NLU output: {intent: get_weather, location: "Paris" }

- ▶ Direct SLU models bypass transcription and predict meaning directly from audio signals.
- ▶ How it works:
 - Input features: log-Mel spectrograms or wav2vec embeddings.
 - Model architecture: typically CNNs or Transformers for classification or sequence labeling.
 - Output: intent + slots, directly mapped from audio.

► Advantages:

- Avoids error propagation from ASR mistakes.
- Ideal for domain-specific commands where transcription is unnecessary.
- Faster and more efficient than pipeline approaches.

► Example:

- Audio: “Play jazz”
- Direct SLU output: {intent: play_music, genre: “jazz” }
- No intermediate transcript is generated.

- ▶ Spoken Language Understanding faces several real-world challenges:
- ▶ **Error Propagation:** In pipeline systems, ASR mistakes can cause NLU errors. *Example:* “Play rock” misheard as “Play doc” → system fails.
- ▶ **Data Scarcity:** Annotated SLU datasets are often small. Collecting paired (speech, intent, slots) data is expensive.
- ▶ **Multilinguality & Code-Switching:** Users often mix languages in the same utterance. *Example:* “Ponme una canción by Shakira.” Systems must handle hybrid language input robustly.
- ▶ **Context & Dialogue:** Understanding multi-turn conversations requires memory of previous interactions. *Example:* “Play jazz.” → “Now make it louder.”

- ▶ Spoken Language Understanding powers many real-world applications by extracting meaning from speech, not just transcribing it.
- ▶ Common use cases:
 - **Virtual assistants:** Siri, Alexa, Google Assistant.
 - **Smart homes:** voice-controlled lights, appliances.
 - **Customer service:** routing calls based on intent.
 - **Automotive:** navigation, calls, climate control.
 - **Healthcare:** converting doctors' spoken notes into structured forms.

► Example (Alexa):

- User says: “Turn off the bedroom lights.”
- SLU output: {intent: control_device, device: lights, location: bedroom}

► Why SLU is critical:

- Transcription alone isn't sufficient; systems must act on the meaning of what was said.

Advanced Tasks: Diarization & Emotion Recognition

► **Goal:** Identify “who spoke when” in audio streams

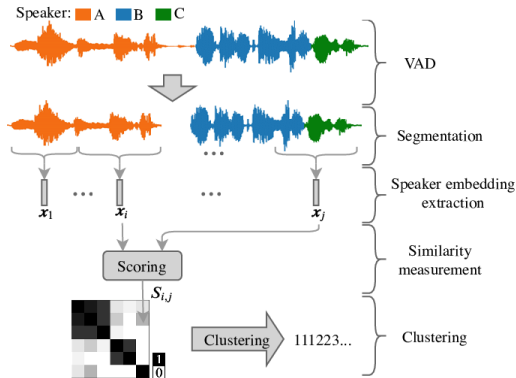
► **Applications:**

- Meeting transcription
- Podcast analysis
- Call center monitoring

► **Pipeline:**

1. **Voice activity detection (VAD):** separate speech from silence and noise
2. **Segmentation:** cut the speech into windows, or detect speaker change points
3. **Embedding:** map each segment to a speaker vector (x-vector, ECAPA-TDNN)
4. **Clustering:** group segments by speaker from pairwise similarities
5. **Resegmentation:** refine the boundaries and assign overlapping speech

VAD comes first for a reason: any speech it drops is lost for good, and any noise it admits gets assigned to some speaker. Its errors propagate through every later stage.

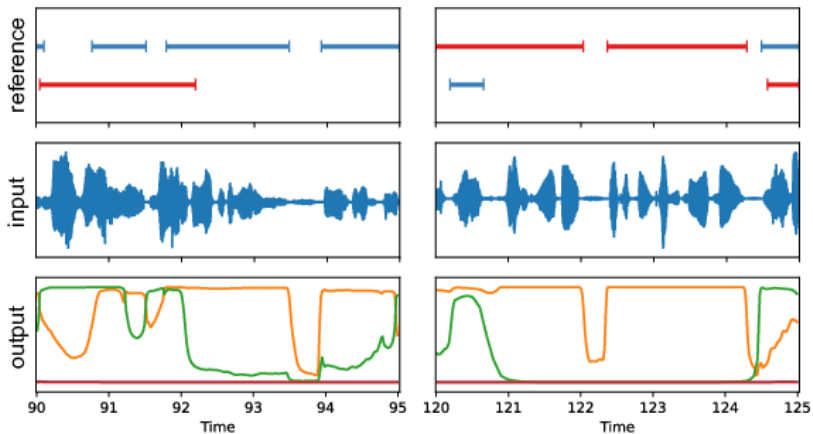


Source: Q. Lin, Y. Hou and M. Li, "Self-Attentive Similarity Measurement Strategies in Speaker Diarization," Interspeech 2020,

Fig. 1.

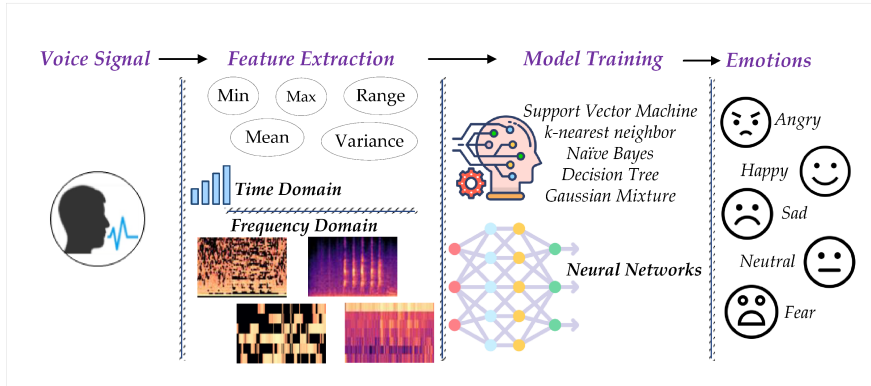
- ▶ **Popular Tool:** `pyannote.audio` (Bredin et al., ICASSP 2020)
- ▶ **Features:**
 - Pre-trained diarization models
 - Easy integration with Python
 - Supports custom pipelines
- ▶ Recent versions replace the modular chain above with an end-to-end neural segmentation model, so the classical pipeline is now one option rather than the only one.

Speaker Diarization Tools (cont.)



Reference annotation (top) against the model's frame-wise speaker posteriors (bottom). Source: `pyannote.audio`.

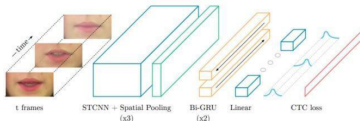
- ▶ **Goal:** Detect emotions from speech signals
- ▶ **Applications:**
 - Human-computer interaction
 - Mental health monitoring
 - Customer service analytics
- ▶ **Datasets:** IEMOCAP, RAVDESS, MSP-IMPROV, CREMA-D



The classical SER pipeline. Source: A. S. Alluhaidan, O. Saidani, R. Jahangir, M. A. Nauman and O. S. Neffati, "Speech Emotion Recognition through Hybrid Features and Convolutional Neural Network," *Applied Sciences* 13(8):4750, 2023, Fig. 1 (CC BY 4.0).

- ▶ The standard recipe is a **self-supervised speech encoder plus a light classifier head**:
 - **Wav2Vec 2.0, HuBERT, WavLM**: general-purpose encoders, robust to noise and speaker variation
 - **emotion2vec**: pre-trained on emotion data specifically
 - Frames are pooled (mean or attention pooling) and passed to a small classifier
- ▶ **Performance on IEMOCAP** (4-class: angry, happy, neutral, sad):
 - ~63–68% accuracy with the encoder *frozen* and only a linear head trained
 - ~73–75% once the encoder itself is *fine-tuned*
- ▶ **Compare numbers with care.** IEMOCAP results are almost always reported on the 4-class collapse with “excited” merged into “happy”, under leave-one-session-out splits, and papers differ on whether they quote weighted (WA) or unweighted (UA) accuracy. Two figures from different papers are usually not comparable.

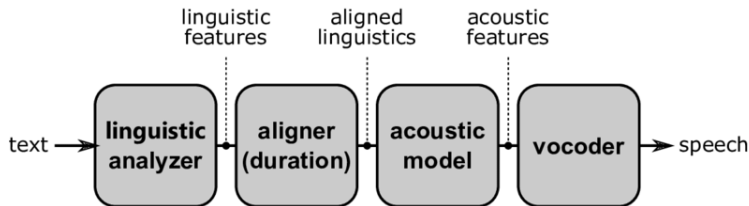
- ▶ **LipNet** (Assael et al., 2016) decodes sentences from lip movement alone, with no audio at all.
- ▶ **Architecture:** spatiotemporal convolutions over the video frames → bidirectional GRU → CTC loss, trained end to end. The same CTC machinery as in audio ASR, with video frames in place of spectrogram frames.
- ▶ **Result:** 95.2% sentence-level accuracy on the GRID corpus (overlapped-speaker split), against 86.4% for the previous word-level state of the art.
- ▶ **Why it matters:** it gives a channel that survives when the audio does not. Fusing the two, *audio-visual* speech recognition, improves accuracy in noise.



Source: Assael et al., "LipNet: End-to-End Sentence-level Lipreading", 2016, Fig. 1.

- ▶ Speech synthesis (Text-to-Speech, TTS) converts written text into spoken audio: the inverse problem of ASR.
- ▶ **Pipeline:** text analysis $\rightarrow$ acoustic model (text to mel-spectrogram) $\rightarrow$ vocoder (mel-spectrogram to waveform). Almost every named system below is one of those two stages, not a whole system.
- ▶ **Three eras:**
 - concatenative / unit-selection and statistical-parametric (HMM-based) synthesis;
 - autoregressive neural: Tacotron 2 (acoustic model) driving a WaveNet vocoder;
 - parallel and diffusion neural: FastSpeech 2 (non-autoregressive acoustic model) or Grad-TTS (diffusion acoustic model), paired with a parallel vocoder such as HiFi-GAN.
- ▶ These pair up rather than compete: Grad-TTS in fact uses HiFi-GAN as its vocoder. Fully end-to-end systems such as VITS collapse both stages into one model.

- **Applications:** accessibility and screen readers, voice assistants, audiobooks, navigation, language learning.



Source: H.-T. Luong, "Deep learning based voice cloning framework for a unified system of text-to-speech and voice conversion," PhD thesis, SOKENDAI, 2020, Fig. 2.1.

- ▶ Modern TTS systems face several practical and ethical challenges:
- ▶ **Prosody & Emotion:** Generating natural intonation, emphasis, and expressive speech remains difficult.
- ▶ **Cross-Speaker Synthesis:** Voice cloning or multi-speaker synthesis requires adapting models to new voices without losing quality.
- ▶ **Low-Resource Languages:** Limited data makes it challenging to build high-quality TTS systems for many languages.
- ▶ **Ethical Issues:** Risk of misuse in deepfakes or generating fake celebrity voices.

Evaluation

- ▶ Word Error Rate (WER) is a standard metric to evaluate ASR transcription accuracy. It is based on the edit distance between the reference and the hypothesis.
- ▶ Formula:

$$\text{WER} = \frac{S + D + I}{N}$$

where:

- S = substitutions
- D = deletions
- I = insertions
- N = total words in reference

► Example:

- Reference: “the cat sat on the mat”
- Hypothesis: “the cat sat mat”
- Errors: 2 deletions $\rightarrow \text{WER} = 2/6 \approx 33.3\%$

► Why it matters:

- Provides an intuitive, widely-used measure of transcription accuracy for ASR systems.

► Two things to keep in mind:

- The denominator counts *reference* words, so insertions are unbounded and **WER can exceed 100%**. A model that hallucinates a paragraph over a one-word utterance scores far above 1.
- WER is computed after normalisation (case, punctuation, numbers), and different normalisers give different WERs on the same output. Comparisons across papers are only meaningful when the normaliser matches.

WER answers one question: did we get the words right. A diarization system with excellent WER can still be useless if it attributes every turn to the wrong speaker, and a synthesizer can be word-perfect and still sound wrong. Each task needs its own metric.

- **CER (Character Error Rate)** is the same edit distance counted over characters instead of words:

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c}$$

Used where word boundaries are not marked in the writing system (Chinese, Japanese, Thai), and for character-level output models, where a one-character slip should not cost a whole word.

- **DER (Diarization Error Rate)** scores “who spoke when”:

$$\text{DER} = \frac{T_{\text{false alarm}} + T_{\text{missed}} + T_{\text{confusion}}}{T_{\text{total speech}}}$$

that is, non-speech scored as speech, plus speech scored as non-speech, plus speech given to the wrong speaker, over total reference speech time. Like WER it can exceed 100%. Always state whether a forgiveness *collar* (commonly 0.25 s) was applied around reference boundaries and whether overlapping speech was scored: DER is not comparable across those settings.

- ▶ **MOS (Mean Opinion Score)** is the reference metric for synthesis, and it is a *human* measurement. Listeners rate each utterance in isolation on a five-point scale, 5 excellent, 4 good, 3 fair, 2 poor, 1 bad, and the score is the mean over ratings (ITU-T Rec. P.800). TTS work adapts the scale to ask about **naturalness** rather than transmission quality.
 - MOS is not comparable across studies: report the number of listeners, the number of utterances, and confidence intervals.
 - Good systems now sit close to recorded speech, so plain MOS saturates. Side-by-side comparative MOS (CMOS) or AB preference tests separate them better.
 - Objective proxies exist but do not replace it: mel-cepstral distortion (MCD), which needs time-aligned ground-truth audio and so only works when reconstructing held-out recordings; ASR word error rate on synthesized speech as an intelligibility check; and learned MOS predictors.

Challenges & Current Research

- ▶ **Robustness:** Handling noise, accents, and low-quality microphones.
- ▶ **Multilinguality & Code-Switching:** Supporting diverse populations and hybrid language input.
- ▶ **Low-Resource Speech:** Many languages lack sufficient training data for ASR/TTS models.
- ▶ **Expressive Speech Synthesis:** Capturing prosody, emotion, and enabling style transfer.
- ▶ **Evaluation:** Moving beyond BLEU/WER to semantic, human-centered, and LLM-based metrics.
- ▶ **Ethics:** Addressing risks like deepfake voices and protecting privacy in speech datasets.

- ▶ Speech is a rich modality for natural language processing, bridging sound and meaning.
- ▶ Audio features such as **MFCC**, **Mel**, and **Log-Mel** form the foundation for ASR and TTS systems.
- ▶ The spoken interaction cycle involves **ASR** → **SLU** → **TTS**, enabling complete end-to-end communication.
- ▶ **Evaluation**: WER measures transcription accuracy for ASR; human listening tests remain the reference for synthesis quality.
- ▶ Future directions:
 - More **end-to-end models** for robust and expressive speech understanding/synthesis.
 - Improved, **semantic and human-centered evaluation metrics**.
 - Stronger **ethical safeguards** against misuse.

References

- [1] Jurafsky, D., & Martin, J. H. (2026). *Speech & Language Processing* (3rd edition draft).
<https://web.stanford.edu/~jurafsky/slp3/>
- [2] Rabiner, L., & Juang, B.-H. (1993). *Fundamentals of Speech Recognition*. Prentice Hall.
- [3] Deng, L., & Li, X. (2013). Machine learning paradigms for speech recognition: An overview. *IEEE Transactions on Audio, Speech, and Language Processing*, 21(5), 1060–1089.
- [4] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- [5] Kubichek, R. F. (1993). Mel-cepstral distance measure for objective speech quality assessment. *IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM)*, 125–128.
- [6] Rabiner, L. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2), 257–286.

- [7] Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006). Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. *ICML*, 369–376.
- [8] Hannun, A. (2017). Sequence modeling with CTC. *Distill*. <https://distill.pub/2017/ctc/>
- [9] Graves, A. (2012). Sequence transduction with recurrent neural networks. *arXiv preprint arXiv:1211.3711*.
- [10] Chan, W., Jaitly, N., Le, Q. V., & Vinyals, O. (2016). Listen, attend and spell: A neural network for large vocabulary conversational speech recognition. *ICASSP*, 4960–4964. arXiv:1508.01211.
- [11] Park, D. S., Chan, W., Zhang, Y., Chiu, C.-C., Zoph, B., Cubuk, E. D., & Le, Q. V. (2019). SpecAugment: A simple data augmentation method for automatic speech recognition. *Interspeech*, 2613–2617. arXiv:1904.08779.
- [12] Gulati, A., Qin, J., Chiu, C.-C., et al. (2020). Conformer: Convolution-augmented transformer for speech recognition. *Interspeech*, 5036–5040. arXiv:2005.08100.

- [13] Rekish, D., Koluguri, N. R., Krizan, S., et al. (2023). Fast Conformer with linearly scalable attention for efficient speech recognition. *IEEE ASRU*, 1–8. arXiv:2305.05084.
- [14] Majumdar, S., Balam, J., Hrinchuk, O., Lavrukhin, V., Noroozi, V., & Ginsburg, B. (2021). Citrinet: Closing the gap between non-autoregressive and autoregressive end-to-end models for automatic speech recognition. *arXiv preprint arXiv:2104.01721*.
- [15] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*.
- [16] Baevski, A., Zhou, H., Mohamed, A., & Auli, M. (2020). wav2vec 2.0: A framework for self-supervised learning of speech representations. *NeurIPS 33*. arXiv:2006.11477.
- [17] Hsu, W.-N., Bolte, B., Tsai, Y.-H. H., Lakhota, K., Salakhutdinov, R., & Mohamed, A. (2021). HuBERT: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29, 3451–3460. arXiv:2106.07447.

- [18] Chen, S., Wang, C., Chen, Z., et al. (2022). WavLM: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6), 1505–1518. arXiv:2110.13900.
- [19] Wang, Y., et al. (2017). Tacotron: Towards end-to-end speech synthesis. *arXiv preprint arXiv:1703.10135*.
- [20] Shen, J., Pang, R., Weiss, R. J., et al. (2018). Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions. *ICASSP*, 4779–4783. arXiv:1712.05884.
- [21] van den Oord, A., et al. (2016). WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*.
- [22] Ren, Y., Hu, C., Tan, X., Qin, T., Zhao, S., Zhao, Z., & Liu, T.-Y. (2021). FastSpeech 2: Fast and high-quality end-to-end text to speech. *ICLR*. arXiv:2006.04558.
- [23] Kong, J., Kim, J., & Bae, J. (2020). HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis. *NeurIPS 33*. arXiv:2010.05646.
- [24] Popov, V., Vovk, I., Gogoryan, V., Sadekova, T., & Kudinov, M. (2021). Grad-TTS: A diffusion probabilistic model for text-to-speech. *ICML*, PMLR 139, 8599–8608. arXiv:2105.06337.

- [25] Kim, J., Kong, J., & Son, J. (2021). VITS: Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. *ICML*. arXiv:2106.06103.
- [26] Park, T. J., Kanda, N., Dimitriadis, D., Han, K. J., Watanabe, S., & Narayanan, S. (2022). A review of speaker diarization: Recent advances with deep learning. *Computer Speech & Language*, 72, 101317. arXiv:2101.09624.
- [27] Lin, Q., Hou, Y., & Li, M. (2020). Self-attentive similarity measurement strategies in speaker diarization. *Interspeech*, 284–288.
- [28] Bredin, H., Yin, R., Coria, J. M., et al. (2020). pyannote.audio: Neural building blocks for speaker diarization. *ICASSP*, 7124–7128.
- [29] Busso, C., Bulut, M., Lee, C.-C., et al. (2008). IEMOCAP: Interactive emotional dyadic motion capture database. *Language Resources and Evaluation*, 42(4), 335–359.
- [30] Chen, L.-W., & Rudnicky, A. (2023). Exploring wav2vec 2.0 fine tuning for improved speech emotion recognition. *ICASSP*, 1–5. arXiv:2110.06309.

- [31] Ma, Z., Zheng, Z., Ye, J., Li, J., Gao, Z., Zhang, S., & Chen, X. (2024). emotion2vec: Self-supervised pre-training for speech emotion representation. *Findings of ACL 2024*, 15747–15760. arXiv:2312.15185.
- [32] Assael, Y. M., Shillingford, B., Whiteson, S., & de Freitas, N. (2016). LipNet: End-to-end sentence-level lipreading. *arXiv preprint arXiv:1611.01599*.
- [33] Chung, J. S., Senior, A., Vinyals, O., & Zisserman, A. (2017). Lip reading sentences in the wild. *CVPR*. arXiv:1611.05358.
- [34] Ravanelli, M., et al. (2021). SpeechBrain: A general-purpose speech toolkit. *arXiv preprint arXiv:2106.04624*.
- [35] NVIDIA Riva, ASR models reference. <https://docs.nvidia.com/deeplearning/riva/user-guide/docs/reference/models/asr.html>
- [36] ITU-T Recommendation P.800 (1996). *Methods for Subjective Determination of Transmission Quality*. International Telecommunication Union, Geneva.
- [37] Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: A method for automatic evaluation of machine translation. *ACL*.

- [38] Panayotov, V., Chen, G., Povey, D., & Khudanpur, S. (2015). LibriSpeech: An ASR corpus based on public domain audio books. *ICASSP*.
- [39] Ardila, R., et al. (2020). Common Voice: A massively-multilingual speech corpus. *LREC*, 4218–4222. arXiv:1912.06670.
- [40] Hernandez, F., Nguyen, V., Ghannay, S., Tomashenko, N., & Estève, Y. (2018). TED-LIUM 3: Twice as much data and corpus repartition for experiments on speaker adaptation. *Speech and Computer (SPECOM)*, LNCS 11096, 198–208. Springer.
- [41] Carletta, J., et al. (2005). The AMI meeting corpus: A pre-announcement. *MLMI*, LNCS 3869, 28–39. Springer.
- [42] LJSpeech dataset. <https://keithito.com/LJ-Speech-Dataset/>
- [43] CSTR VCTK Corpus (version 0.92). <https://datashare.ed.ac.uk/handle/10283/3443>
- [44] Bakhturina, E., Lavrukhin, V., Ginsburg, B., & Zhang, Y. (2021). Hi-Fi multi-speaker English TTS dataset. *arXiv preprint arXiv:2104.01497*. Download: <https://www.openslr.org/109/>

- [45] ATIS intent/slot dataset. <https://www.kaggle.com/datasets/siddhadev/ms-cntk-atis>
- [46] SLURP dataset (audio hosted on Zenodo). <https://github.com/pswietojski/slurp>
- [47] Whisper ASR Demo. <https://huggingface.co/spaces/openai/whisper>
- [48] Tacotron 2 + WaveGlow Demo.
<https://colab.research.google.com/github/NVIDIA/DeepLearningExamples/blob/master/PyTorch/SpeechSynthesis/Tacotron2/notebooks/Tacotron2.ipynb>
- [49] Google Cloud Text-to-Speech. <https://cloud.google.com/text-to-speech>
- [50] Coqui TTS (maintained fork). <https://github.com/idiap/coqui-ai-TTS>
- [51] Li, J. (2022). Recent advances in end-to-end automatic speech recognition. *APSIPA Transactions on Signal and Information Processing*, 11(1), e8. arXiv:2111.01690.
- [52] Prabhavalkar, R., Hori, T., Sainath, T. N., Schlüter, R., & Watanabe, S. (2024). End-to-end speech recognition: A survey. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 32, 325–351. arXiv:2303.03329.

- [53] Qin, L., Xie, T., Che, W., & Liu, T. (2021). A survey on spoken language understanding: Recent advances and new frontiers. *IJCAI*, 4577–4584. [arXiv:2103.03095](#).
- [54] Tan, X., Qin, T., Soong, F., & Liu, T.-Y. (2021). A survey on neural speech synthesis. *arXiv preprint arXiv:2106.15561*.